

JAVA

Complete Concepts Guide

All Core Java Concepts Covered

Plain Text Edition

TABLE OF CONTENTS

1. Introduction to Java
2. Data Types and Variables
3. Operators
4. Control Flow Statements
5. Arrays
6. Object-Oriented Programming (OOP)
7. Classes and Objects
8. Inheritance
9. Polymorphism
10. Abstraction and Interfaces
11. Encapsulation
12. Constructors
13. Static and Final Keywords
14. Packages and Access Modifiers
15. Exception Handling
16. Strings
17. Collections Framework
18. Generics
19. Iterators and for-each
20. Java I/O
21. Multithreading
22. Lambda Expressions
23. Streams API
24. Functional Interfaces
25. Java Memory Management and JVM

1. INTRODUCTION TO JAVA

Java is a high-level, class-based, object-oriented programming language designed to have as few implementation dependencies as possible. It was developed by James Gosling at Sun Microsystems in 1995. Java follows the principle of Write Once, Run Anywhere (WORA), meaning compiled Java code can run on all platforms that support Java without recompilation.

Key Features:

Platform Independent: Java code is compiled into bytecode that runs on the Java Virtual Machine (JVM), making it OS-independent.

Object-Oriented: Everything in Java is an object. Java supports OOP concepts like inheritance, polymorphism, abstraction, and encapsulation.

Strongly Typed: Every variable and expression has a type, and types are checked at compile time.

Automatic Memory Management: Java uses garbage collection to automatically manage memory.

Multithreaded: Java supports multithreading at the language level.

Robust and Secure: Java has strong type checking, exception handling, and a security manager.

Java Program Structure:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Every Java program starts execution from the main() method. The class name must match the filename. Java files have the .java extension; compiled files have the .class extension.

How Java Works:

Source Code (.java) -> Compiler (javac) -> Bytecode (.class) -> JVM -> Machine Code -> Output. The JVM interprets bytecode and translates it to machine-specific instructions at runtime.

2. DATA TYPES AND VARIABLES

Java has two categories of data types: Primitive and Reference (Non-Primitive).

Primitive Data Types:

byte: 8-bit integer. Range: -128 to 127. Default: 0.

short: 16-bit integer. Range: -32,768 to 32,767. Default: 0.

int: 32-bit integer. Range: -2^{31} to $2^{31}-1$. Default: 0.

long: 64-bit integer. Range: -2^{63} to $2^{63}-1$. Default: 0L.

float: 32-bit floating point. Default: 0.0f.

double: 64-bit floating point. Default: 0.0d.

char: 16-bit Unicode character. Range: 0 to 65,535. Default: '\u0000'.

boolean: true or false. Default: false.

Reference Types:

Reference types include Strings, Arrays, Classes, and Interfaces. They store references (addresses) to the actual data in heap memory.

Variable Declaration and Initialization:

```
int age = 25;
double salary = 55000.50;
char grade = 'A';
boolean isActive = true;
String name = "Java";
```

Type Casting:

Widening (Implicit): byte -> short -> int -> long -> float -> double. Done automatically.

Narrowing (Explicit): Requires explicit cast. May lose data.

```
int x = 100;
long y = x;           // Widening - automatic
int z = (int) y;      // Narrowing - explicit cast
```

var keyword (Java 10+):

```
var list = new ArrayList(); // Type inferred as ArrayList
var count = 10;              // Type inferred as int
```

3. OPERATORS

Arithmetic Operators: + (addition), - (subtraction), * (multiplication), / (division), % (modulus), ++ (increment), -- (decrement).

Relational Operators:

== (equal), != (not equal), > (greater), < (less), >= (greater or equal), <= (less or equal). Return boolean values.

Logical Operators:

&& (AND): true if both operands are true. || (OR): true if at least one operand is true. ! (NOT): reverses boolean value.

Bitwise Operators:

& (AND), | (OR), ^ (XOR), ~ (complement), << (left shift), >> (right shift), >>> (unsigned right shift).

Assignment Operators:

= (assign), += (add and assign), -= (subtract and assign), *= (multiply and assign), /= (divide and assign), %= (modulus and assign).

Ternary Operator:

```
int max = (a > b) ? a : b; // If a > b, max = a, else max = b
```

instanceof Operator:

```
String s = "Hello";  
boolean result = s instanceof String; // true
```

4. CONTROL FLOW STATEMENTS

if-else Statement:

```
if (condition) {  
    // block 1  
} else if (condition2) {  
    // block 2  
} else {  
    // default block  
}
```

switch Statement:

```
switch (day) {  
    case 1: System.out.println("Monday"); break;  
    case 2: System.out.println("Tuesday"); break;  
    default: System.out.println("Other");  
}
```

Switch Expressions (Java 14+): The new switch expression can return a value using the arrow syntax.

```
String result = switch (day) {  
    case 1 -> "Monday";  
    case 2 -> "Tuesday";  
    default -> "Other";  
};
```

Loops:

for loop: Used when number of iterations is known.

```
for (int i = 0; i < 10; i++) {  
    System.out.println(i);  
}
```

while loop: Used when condition is checked before each iteration.

```
while (i < 10) {  
    i++;  
}
```

do-while loop: Executes at least once, checks condition after.

```
do {  
    i++;  
} while (i < 10);
```

break and continue:

break: Exits the loop or switch immediately. continue: Skips the current iteration and moves to the next.

Labeled break/continue allows breaking out of nested loops.

```
outer:  
for (int i = 0; i < 5; i++) {  
    for (int j = 0; j < 5; j++) {  
        if (j == 2) break outer; // Breaks both loops  
    }  
}
```

5. ARRAYS

An array is a fixed-size sequential collection of elements of the same type. Arrays are objects in Java.

Declaration and Initialization:

```
int[] arr = new int[5];           // Declare with size
int[] arr2 = {1, 2, 3, 4, 5};    // Declare with values
String[] names = new String[3];
```

Accessing Elements:

```
arr[0] = 10; // Assign
int x = arr[0]; // Access
int len = arr.length; // Get length
```

Multi-dimensional Arrays:

```
int[][] matrix = new int[3][3];
int[][] grid = {{1,2,3},{4,5,6},{7,8,9}};
System.out.println(grid[1][2]); // Output: 6
```

Iterating Arrays:

```
// Traditional for loop
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
// Enhanced for loop
for (int val : arr) {
    System.out.println(val);
}
```

Arrays Utility Class:

```
import java.util.Arrays;
Arrays.sort(arr);           // Sort array
Arrays.fill(arr, 0);        // Fill all with 0
int idx = Arrays.binarySearch(arr, 5); // Binary search
int[] copy = Arrays.copyOf(arr, 3);    // Copy first 3 elements
System.out.println(Arrays.toString(arr)); // Print nicely
```

6. OBJECT-ORIENTED PROGRAMMING (OOP)

OOP is a programming paradigm that organizes software design around data (objects) rather than functions and logic. Java is a fully object-oriented language.

The Four Pillars of OOP:

1. Encapsulation: Bundling data (fields) and methods that operate on the data into a single unit (class), and restricting direct access to the internal state.
2. Inheritance: A mechanism where a new class (child) derives properties and behaviors from an existing class (parent). Promotes code reusability.
3. Polymorphism: The ability of an object to take many forms. The same method or operator behaves differently based on the context.
4. Abstraction: Hiding the implementation details and showing only the functionality to the user. Achieved using abstract classes and interfaces.

Benefits of OOP:

Code reusability through inheritance. Modularity and maintainability. Data security through encapsulation. Flexibility through polymorphism. Easy to model real-world problems.

7. CLASSES AND OBJECTS

A class is a blueprint or template from which objects are created. An object is an instance of a class.

Defining a Class:

```
public class Car {  
    // Fields (instance variables)  
    String brand;  
    int speed;  
    // Method  
    void accelerate() {  
        speed += 10;  
        System.out.println(brand + " speed: " + speed);  
    }  
}
```

Creating Objects:

```
Car myCar = new Car(); // Instantiation  
myCar.brand = "Toyota";  
myCar.speed = 0;  
myCar.accelerate(); // Output: Toyota speed: 10
```

this Keyword:

Refers to the current object. Used to differentiate instance variables from parameters with the same name.

```
public class Person {  
    String name;  
    void setName(String name) {  
        this.name = name; // this.name is instance var  
    }  
}
```

Instance vs Class Members:

Instance variables/methods belong to an object. Static variables/methods belong to the class itself.

8. INHERITANCE

Inheritance enables a class (subclass/child) to acquire the properties and behaviors of another class (superclass/parent). Java supports single inheritance for classes but multiple inheritance through interfaces.

extends Keyword:

```
public class Animal {
    String name;
    void eat() { System.out.println(name + " eats"); }
}

public class Dog extends Animal {
    void bark() { System.out.println(name + " barks"); }
}

Dog d = new Dog();
d.name = "Rex";
d.eat();    // Inherited method
d.bark();   // Own method
```

super Keyword:

super refers to the parent class. Used to call the parent's constructor or methods.

```
public class Dog extends Animal {
    Dog(String name) {
        super(name); // Call parent constructor
    }
    void display() {
        super.eat(); // Call parent method
    }
}
```

Method Overriding:

A subclass provides its own implementation of a method defined in the parent class. Use @Override annotation.

```
class Animal {
    void sound() { System.out.println("Some sound"); }
}
class Cat extends Animal {
    @Override
    void sound() { System.out.println("Meow"); }
}
```

Types of Inheritance in Java:

Single: A extends B. Multilevel: A extends B, B extends C. Hierarchical: Multiple classes extend the same parent. Multiple: Achieved only through interfaces.

9. POLYMORPHISM

Polymorphism means 'many forms'. In Java, it allows one interface to be used for different data types or methods.

Compile-Time Polymorphism (Method Overloading):

Same method name with different parameters in the same class.

```
class MathHelper {  
    int add(int a, int b) { return a + b; }  
    double add(double a, double b) { return a + b; }  
    int add(int a, int b, int c) { return a + b + c; }  
}
```

Runtime Polymorphism (Method Overriding):

The correct method is determined at runtime based on the actual object type.

```
Animal a = new Dog(); // Upcasting  
a.sound(); // Calls Dog's sound(), resolved at runtime
```

Upcasting and Downcasting:

```
Animal a = new Dog(); // Upcasting (automatic)  
Dog d = (Dog) a; // Downcasting (explicit)  
// Check before casting:  
if (a instanceof Dog) {  
    Dog dog = (Dog) a;  
}
```

Covariant Return Types:

An overriding method in a child class can return a subtype of the return type declared in the parent method.

10. ABSTRACTION AND INTERFACES

Abstract Classes:

Cannot be instantiated. Can have abstract (no body) and concrete (with body) methods. Used when classes share common methods but have different implementations.

```
abstract class Shape {
    abstract double area();    // Must be overridden
    void display() {           // Can be used directly
        System.out.println("Area: " + area());
    }
}
class Circle extends Shape {
    double radius;
    Circle(double r) { radius = r; }
    double area() { return Math.PI * radius * radius; }
}
```

Interfaces:

A blueprint of a class. All methods are public and abstract by default (pre-Java 8). Fields are public, static, and final. A class can implement multiple interfaces.

```
interface Flyable {
    void fly(); // abstract by default
}
interface Swimmable {
    void swim();
}
class Duck implements Flyable, Swimmable {
    public void fly() { System.out.println("Duck flies"); }
    public void swim() { System.out.println("Duck swims"); }
}
```

Default and Static Methods in Interfaces (Java 8+):

```
interface Greetable {
    default void greet() { System.out.println("Hello!"); }
    static void info() { System.out.println("Interface info"); }
}
```

Abstract Class vs Interface:

Abstract class can have constructors and state; interfaces cannot (before Java 9). A class can extend only one abstract class but implement many interfaces. Use abstract class for IS-A; interface for CAN-DO relationships.

11. ENCAPSULATION

Encapsulation is wrapping data and the methods that operate on it into a single unit and restricting outside access. Implemented using private fields and public getters/setters.

```
public class BankAccount {
    private double balance; // Private field
    private String owner;

    public BankAccount(String owner, double balance) {
        this.owner = owner;
        this.balance = balance;
    }
    // Getter
    public double getBalance() { return balance; }
    // Setter with validation
    public void deposit(double amount) {
        if (amount > 0) balance += amount;
    }
    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance)
            balance -= amount;
    }
}
```

Benefits of Encapsulation:

Data hiding: Internal state is not directly accessible. Validation: You control what values are set. Flexibility: Implementation can change without affecting outside code. Easy to maintain and test.

12. CONSTRUCTORS

A constructor is a special method used to initialize objects. It has the same name as the class and no return type.

Types of Constructors:

Default Constructor: No parameters. Provided by Java if no constructor defined.

Parameterized Constructor: Takes parameters to initialize fields.

Copy Constructor: Takes an object of the same class as a parameter.

```
public class Student {  
    String name;  
    int age;  
    // Default  
    Student() { name = "Unknown"; age = 0; }  
    // Parameterized  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
    // Copy  
    Student(Student s) {  
        this.name = s.name;  
        this.age = s.age;  
    }  
}
```

Constructor Chaining:

Using this() to call another constructor of the same class. Using super() to call a parent class constructor.

```
Student() {  
    this("Unknown", 0); // Calls parameterized constructor  
}
```

13. STATIC AND FINAL KEYWORDS

static Keyword:

Static belongs to the class, not to any object. Static variables are shared among all objects. Static methods can be called without creating an object.

```
class Counter {
    static int count = 0; // Shared across all objects
    Counter() { count++; }
    static int getCount() { return count; }
}
Counter c1 = new Counter();
Counter c2 = new Counter();
System.out.println(Counter.getCount()); // 2
```

Static Initializer Block:

```
class Config {
    static String dbUrl;
    static {
        dbUrl = "jdbc:mysql://localhost/mydb"; // Runs once
    }
}
```

final Keyword:

final variable: Cannot be reassigned (constant).

final method: Cannot be overridden.

final class: Cannot be extended (e.g., String class).

```
final double PI = 3.14159; // Constant
final class ImmutableClass { ... } // Cannot be subclassed
final void show() { ... } // Cannot be overridden
```

static final (Constants):

```
public static final double MAX_SPEED = 300.0;
```

14. PACKAGES AND ACCESS MODIFIERS

Packages:

A package is a namespace that organizes classes and interfaces. It prevents naming conflicts and controls access. Use import to access classes from other packages.

```
package com.myapp.model; // Declare package
import java.util.ArrayList; // Import specific class
import java.util.*; // Import all classes in package
```

Access Modifiers:

public: Accessible from anywhere.

protected: Accessible within same package and subclasses.

default (no modifier): Accessible only within same package.

private: Accessible only within the same class.

Access Modifier Scope Table:

Modifier	Same Class	Same Package	Subclass	Other Package
public	YES	YES	YES	YES
protected	YES	YES	YES	NO
default	YES	YES	NO	NO
private	YES	NO	NO	NO

15. EXCEPTION HANDLING

Exception handling is a mechanism to handle runtime errors, maintaining normal program flow. Java uses try-catch-finally blocks.

Exception Hierarchy:

Throwable -> Error (JVM errors, unchecked) and Exception. Exception -> RuntimeException (unchecked) and Checked Exceptions.

Checked exceptions (e.g., IOException, SQLException) must be declared or handled. Unchecked exceptions (e.g., NullPointerException, ArrayIndexOutOfBoundsException) don't need to be declared.

try-catch-finally:

```
try {
    int result = 10 / 0; // ArithmeticException
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
} catch (Exception e) {
    System.out.println("General error");
} finally {
    System.out.println("Always executes");
}
```

throws and throw:

```
// throws: declares method may throw exception
void readFile(String path) throws IOException {
    // ...
}
// throw: explicitly throws an exception
void validate(int age) {
    if (age < 0) throw new IllegalArgumentException("Invalid age");
}
```

Custom Exceptions:

```
class InsufficientFundsException extends Exception {
    InsufficientFundsException(String message) {
        super(message);
    }
}
```

try-with-resources (Java 7+):

Automatically closes resources that implement AutoCloseable.

```
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    String line = br.readLine();
} // br is automatically closed
```

16. STRINGS

Strings in Java are immutable objects of the `String` class. Once created, their values cannot be changed.

Creating Strings:

```
String s1 = "Hello";           // String literal (String pool)
String s2 = new String("Hello"); // New object in heap
char[] chars = {'J', 'a', 'v', 'a'};
String s3 = new String(chars);
```

Common String Methods:

```
String s = "Hello, World!";
s.length()           // 13
s.charAt(0)          // 'H'
s.indexOf("World")   // 7
s.substring(7)       // "World!"
s.substring(7, 12)   // "World"
s.toLowerCase()      // "hello, world!"
s.toUpperCase()       // "HELLO, WORLD!"
s.trim()             // Remove leading/trailing spaces
s.replace('l', 'r')   // "Herro, Worrd!"
s.contains("World")   // true
s.startsWith("He")    // true
s.split(", ")        // ["Hello", "World!"]
```

String Comparison:

```
s1.equals(s2)          // Compare content
s1.equalsIgnoreCase(s2) // Case-insensitive compare
s1.compareTo(s2)        // Lexicographic comparison
// Never use == for String content comparison!
```

StringBuilder and StringBuffer:

Mutable alternatives to `String`. `StringBuilder` is faster (not thread-safe); `StringBuffer` is thread-safe.

```
StringBuilder sb = new StringBuilder();
sb.append("Hello");
sb.append(" World");
sb.insert(5, ",");
sb.delete(0, 5);
String result = sb.toString();
```

String.format() and String Interpolation:

```
String msg = String.format("Name: %s, Age: %d", "Alice", 30);
// Java 15+ Text Blocks:
String json = """
    {"name": "Alice",
      "age": 30}
    """;
```

17. COLLECTIONS FRAMEWORK

The Java Collections Framework provides a set of classes and interfaces to store and manipulate groups of objects. Main interfaces: Collection, List, Set, Queue, Map.

List:

Ordered collection. Allows duplicates. Main implementations: ArrayList (dynamic array, fast access) and LinkedList (doubly linked, fast insert/delete).

```
List list = new ArrayList<>();
list.add("Apple");
list.add("Banana");
list.get(0);           // "Apple"
list.remove("Apple");
list.size();           // 1
Collections.sort(list);
```

Set:

No duplicates. HashSet (unordered, fast), LinkedHashSet (insertion order), TreeSet (sorted).

```
Set set = new HashSet<>();
set.add(1); set.add(2); set.add(1); // Duplicate ignored
System.out.println(set.size()); // 2
```

Map:

Key-value pairs. No duplicate keys. HashMap (unordered), LinkedHashMap (insertion order), TreeMap (sorted by key).

```
Map map = new HashMap<>();
map.put("Alice", 90);
map.put("Bob", 85);
map.get("Alice");           // 90
map.containsKey("Bob"); // true
map.remove("Bob");
for (Map.Entry e : map.entrySet()) {
    System.out.println(e.getKey() + ": " + e.getValue());
}
```

Queue and Deque:

```
Queue queue = new LinkedList<>();
queue.offer("First"); // Add
queue.poll();         // Remove and return head
queue.peek();         // View head without removing
Deque deque = new ArrayDeque<>(); // Double-ended queue
```

Stack:

```
Deque stack = new ArrayDeque<>();
stack.push(1); stack.push(2);
stack.pop();    // Returns 2 (LIFO)
stack.peek();   // Returns top without removing
```

18. GENERICS

Generics allow types (classes and interfaces) to be parameters when defining classes, interfaces, and methods. They provide type safety and eliminate the need for casting.

Generic Class:

```
public class Box {
    private T value;
    public Box(T value) { this.value = value; }
    public T get() { return value; }
}
Box strBox = new Box<>("Hello");
Box intBox = new Box<>(42);
String s = strBox.get(); // No casting needed
```

Generic Methods:

```
public void printArray(T[] arr) {
    for (T item : arr) System.out.print(item + " ");
}
printArray(new Integer[]{1, 2, 3});
printArray(new String[]{"a", "b", "c"});
```

Bounded Type Parameters:

```
// Upper bounded: T must be Number or its subclass
public double sum(T[] arr) {
    double total = 0;
    for (T t : arr) total += t.doubleValue();
    return total;
}
```

Wildcards:

? (unbounded), ? extends T (upper bounded), ? super T (lower bounded).

```
List list; // Any type
List nums; // Number or subtype
List ints; // Integer or supertype
```

19. ITERATORS AND FOR-EACH

Iterator Interface:

The Iterator interface provides methods to iterate over a collection: hasNext() (returns true if more elements), next() (returns next element), remove() (removes current element).

```
List fruits = Arrays.asList("Apple", "Banana", "Cherry");
Iterator it = fruits.iterator();
while (it.hasNext()) {
    String fruit = it.next();
    System.out.println(fruit);
}
```

Enhanced for-each Loop:

Simplified iteration. Works with any Iterable.

```
for (String fruit : fruits) {
    System.out.println(fruit);
}
```

ListIterator:

Allows bidirectional traversal and modification during iteration.

```
ListIterator li = list.listIterator();
while (li.hasNext()) {
    String s = li.next();
    li.set(s.toUpperCase()); // Modify during iteration
}
```

Iterable Interface:

Any class that implements Iterable can be used in a for-each loop by implementing the iterator() method.

20. JAVA I/O

Java provides comprehensive I/O functionality through the java.io and java.nio packages.

Reading from Console:

```
import java.util.Scanner;
Scanner sc = new Scanner(System.in);
System.out.print("Enter name: ");
String name = sc.nextLine();
int age = sc.nextInt();
sc.close();
```

File I/O with FileReader/FileWriter:

```
// Writing to file
try (FileWriter fw = new FileWriter("out.txt");
    BufferedWriter bw = new BufferedWriter(fw)) {
    bw.write("Hello, File!");
    bw.newLine();
}
// Reading from file
try (FileReader fr = new FileReader("out.txt");
    BufferedReader br = new BufferedReader(fr)) {
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
}
```

Java NIO (New I/O) - Java 7+:

```
import java.nio.file.*;
Path path = Paths.get("data.txt");
// Write
Files.write(path, "Hello NIO".getBytes());
// Read all lines
List lines = Files.readAllLines(path);
// Read entire file as string
String content = Files.readString(path); // Java 11+
```

Serialization:

Converting an object to a byte stream for storage or transmission. Class must implement Serializable.

```
class Person implements Serializable {
    String name; int age;
}
// Serialize
ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("p.ser"));
oos.writeObject(person);
// Deserialize
ObjectInputStream ois = new ObjectInputStream(new FileInputStream("p.ser"));
Person p = (Person) ois.readObject();
```

21. MULTITHREADING

Multithreading allows concurrent execution of two or more threads. Java provides built-in support for multithreading.

Creating Threads:

Method 1: Extend Thread class.

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread: " + Thread.currentThread().getName());
    }
}
new MyThread().start();
```

Method 2: Implement Runnable interface (preferred).

```
class MyTask implements Runnable {
    public void run() { System.out.println("Running"); }
}
Thread t = new Thread(new MyTask());
t.start();
// Or with lambda:
new Thread(() -> System.out.println("Lambda thread")).start();
```

Thread Lifecycle:

New -> Runnable -> Running -> Blocked/Waiting/Sleeping -> Terminated.

Synchronization:

Prevents race conditions when multiple threads access shared resources.

```
class Counter {
    private int count = 0;
    public synchronized void increment() { count++; }
    public synchronized int getCount() { return count; }
}
```

ExecutorService (Thread Pool):

```
ExecutorService executor = Executors.newFixedThreadPool(4);
executor.submit(() -> System.out.println("Task 1"));
executor.submit(() -> System.out.println("Task 2"));
executor.shutdown();
```

volatile Keyword:

Ensures variable reads/writes are directly from/to main memory, not thread cache. Used for flags shared between threads.

```
volatile boolean running = true;
```

22. LAMBDA EXPRESSIONS (Java 8+)

Lambda expressions provide a concise way to represent anonymous functions (functional interfaces). They enable functional programming style in Java.

Syntax: (parameters) -> expression or (parameters) -> { statements }

```
// Traditional anonymous class
Runnable r1 = new Runnable() {
    public void run() { System.out.println("Hello"); }
};
// Lambda equivalent
Runnable r2 = () -> System.out.println("Hello");

// With parameters
Comparator comp = (a, b) -> a.compareTo(b);

// With block body
Comparator byAbs = (a, b) -> {
    int absA = Math.abs(a);
    int absB = Math.abs(b);
    return Integer.compare(absA, absB);
};
```

Using Lambdas with Collections:

```
List names = Arrays.asList("Charlie", "Alice", "Bob");
// Sort with lambda
names.sort((a, b) -> a.compareTo(b));
// Or with method reference
names.sort(String::compareTo);
// forEach
names.forEach(name -> System.out.println(name));
names.forEach(System.out::println); // Method reference
```

Method References:

Static method: ClassName::staticMethod. Instance method: obj::instanceMethod or ClassName::instanceMethod.

Constructor: ClassName::new.

```
Function parser = Integer::parseInt;
Supplier<List> listFactory = ArrayList::new;
```

23. STREAMS API (Java 8+)

The Stream API allows processing of collections in a functional style. A stream is a sequence of elements supporting sequential and parallel aggregate operations.

Creating Streams:

```
// From collection
Stream s1 = list.stream();
// From array
Stream s2 = Arrays.stream(arr);
// From values
Stream s3 = Stream.of("a", "b", "c");
// Infinite stream
Stream ints = Stream.iterate(0, n -> n + 1);
```

Intermediate Operations (return a new Stream):

```
List nums = Arrays.asList(1,2,3,4,5,6,7,8,9,10);
List result = nums.stream()
    .filter(n -> n % 2 == 0)    // Keep even numbers
    .map(n -> n * n)           // Square each
    .sorted()                  // Sort
    .limit(3)                   // Take first 3
    .collect(Collectors.toList());
System.out.println(result);    // [4, 16, 36]
```

Terminal Operations (produce a result):

```
long count = list.stream().filter(s -> s.startsWith("A")).count();
Optional first = list.stream().findFirst();
boolean anyMatch = list.stream().anyMatch(s -> s.length() > 5);
int sum = nums.stream().mapToInt(Integer::intValue).sum();
String joined = list.stream().collect(Collectors.joining(", "));
```

Optional (Java 8+):

A container that may or may not contain a non-null value. Avoids NullPointerException.

```
Optional opt = Optional.of("Hello");
opt.isPresent();           // true
opt.get();                  // "Hello"
opt.orElse("default");     // Returns value or default
opt.ifPresent(System.out::println);
```

24. FUNCTIONAL INTERFACES

A functional interface has exactly one abstract method. They can be used as targets for lambda expressions and method references. Annotate with `@FunctionalInterface`.

Built-in Functional Interfaces (`java.util.function`):

Function: Takes T, returns R. Method: `apply()`.

Consumer: Takes T, returns nothing. Method: `accept()`.

Supplier: Takes nothing, returns T. Method: `get()`.

Predicate: Takes T, returns boolean. Method: `test()`.

BiFunction: Takes T and U, returns R.

UnaryOperator: Takes T, returns T.

BinaryOperator: Takes two T's, returns T.

```
// Function
Function len = String::length;
len.apply("Hello"); // 5

// Predicate
Predicate isEven = n -> n % 2 == 0;
isEven.test(4); // true
isEven.and(n -> n > 0).test(4); // Composing predicates

// Supplier
Supplier greeting = () -> "Hello!";
greeting.get(); // "Hello!"

// Consumer
Consumer printer = System.out::println;
printer.accept("Hello Consumer");
```

Custom Functional Interface:

```
@FunctionalInterface
interface MathOperation {
    int operate(int a, int b);
}

MathOperation add = (a, b) -> a + b;
MathOperation multiply = (a, b) -> a * b;
System.out.println(add.operate(5, 3)); // 8
System.out.println(multiply.operate(5, 3)); // 15
```

25. JAVA MEMORY MANAGEMENT AND JVM

JVM Architecture:

Class Loader: Loads .class files into memory. Bytecode Verifier: Checks code for security and correctness. JIT Compiler: Compiles frequently used bytecode to native machine code at runtime. Garbage Collector: Automatically reclaims unused memory.

JVM Memory Areas:

Heap: Runtime data area where all objects and arrays are stored. Shared across all threads. Garbage collected.

Stack: Per-thread memory. Stores local variables, method call frames, and partial results. LIFO structure.

Method Area (Metaspace in Java 8+): Stores class data, static variables, and constant pool.

Program Counter Register: Per-thread register holding the address of the current instruction.

Native Method Stack: For native methods written in C/C++.

Garbage Collection:

GC automatically reclaims heap memory occupied by objects no longer reachable. Common algorithms: Serial GC, Parallel GC, G1 GC (default since Java 9), ZGC (low-latency, Java 15+).

Objects become eligible for GC when no references point to them.

```
// Suggest GC (no guarantee)
System.gc();
// Make object eligible
MyObj obj = new MyObj();
obj = null; // No reference -> eligible for GC
```

Java 9+ Module System (JPMS):

Modules group related packages with explicit dependencies. Defined in module-info.java.

```
module com.myapp {
    requires java.sql;
    exports com.myapp.service;
}
```

Key JVM Flags:

-Xms: Initial heap size. -Xmx: Maximum heap size. -Xss: Thread stack size. -verbose:gc: Print GC events.

Record Classes (Java 16+):

Compact syntax for immutable data carriers.

```
record Point(int x, int y) {}
Point p = new Point(3, 4);
p.x(); // 3
```

Sealed Classes (Java 17+):

Restricts which classes can extend or implement them.

```
sealed interface Shape permits Circle, Rectangle, Triangle {}
final class Circle implements Shape { ... }
```